

01-Neural-Networks-Fundamentals

1 Neural Networks Fundamentals

Document Version: 1.0

Generated: December 4, 2025

Status: Foundational Reference

1.1 Table of Contents

- [1. Overview](#)
- [2. Neural Network Design Process](#)
- [3. Training Pipeline](#)
- [4. Forward Propagation](#)
- [5. Loss Functions](#)
- [6. Backpropagation Algorithm](#)
- [7. Activation Functions](#)
- [8. Tokenization](#)
- [9. Inference Flow](#)
- [10. Matrix Implementation Notes](#)
- [11. References](#)

1.2 Overview

This document provides an **abstract, implementation-independent** overview of neural network design, training, and inference processes. The goal is to present core algorithmic principles with both mathematical formulations (pseudocode) and practical implementation examples (C++), without diving into platform-specific optimizations (CUDA, GPU kernels, etc.) which are noted as areas for future elaboration.

1.2.1 Purpose & Scope

- Abstract Principles:** Mathematical foundations of neural networks
- Algorithmic Understanding:** Step-by-step processes for training and inference
- Implementation Considerations:** Practical code examples showing how abstract concepts translate to software
- Matrix Representations:** How operations are typically implemented as matrix operations (without optimization details)

1.2.2 Out of Scope

- GPU-specific optimizations (CUDA kernels, cuDNN, etc.)
- Hardware acceleration details (TPUs, specialized AI chips)
- Framework-specific implementations (PyTorch internals, TensorFlow graph optimization)
- Distributed training strategies
- Model compression and quantization

1.3 Conceptual Foundation

1.3.1 What is a Neural Network?

A neural network is fundamentally a **parametric mathematical function** that learns to map inputs to outputs through experience. Imagine teaching a system to recognize cats in photos—rather than explicitly programming rules like “look for pointed ears and whiskers,” we instead provide thousands of cat and non-cat images, and the network automatically discovers the patterns that distinguish them.

The network achieves this through **layers of artificial neurons** (simple mathematical units), where each neuron performs a weighted sum of its inputs and applies a non-linear transformation. These layers are stacked and interconnected, creating a complex function capable of learning intricate patterns in data.

Key intuition: Just as biological neurons fire when they receive sufficient stimulation from their inputs, artificial neurons activate based on weighted combinations of their inputs. The “learning” process adjusts these weights so that the network gradually produces better predictions.

1.3.2 The Learning Problem

Training a neural network addresses a fundamental challenge: **finding the right values for millions of parameters (weights and biases) such that the network makes correct predictions on unseen data.**

This is similar to tuning an instrument—we can't adjust every part simultaneously and hope for the best. Instead, we: 1. Make a prediction (forward pass) 2. Measure how wrong it is (loss calculation) 3. Figure out which adjustments would reduce the error (backpropagation) 4. Make small adjustments in the right direction (gradient descent) 5. Repeat thousands of times

The elegance of neural networks lies in **backpropagation**, an efficient algorithm for computing how each parameter contributes to the total error, enabling us to update millions of parameters in a meaningful way.

1.3.3 Why Non-Linearity Matters

A crucial insight: if we stacked only linear operations (weighted sums), the entire network would remain linear, mathematically equivalent to a single-layer network. This fundamental limitation means we couldn't learn complex, curved decision boundaries.

Non-linear activation functions break this linearity. By applying functions like ReLU or sigmoid after each layer, we enable the network to learn curved, complex relationships between inputs and outputs. This is why activation functions are essential—they're what gives neural networks their expressive power.

For example: - Linear network: Can only draw straight-line decision boundaries between classes - Non-linear network: Can draw arbitrary curved boundaries, allowing it to separate complex patterns

1.4 Neural Network Design Process

1.4.1 High-Level Design Flow

Neural Network Design Process:

1. Problem Definition
2. Architecture Selection
3. Layer Configuration
4. Activation Functions
5. Loss Function Choice
6. Optimizer Selection
7. Training
8. Performance Evaluation → If not acceptable: return to Hyperparameter Tuning → return to Training
9. If acceptable: Model Export
10. Deployment

1.4.2 Design Considerations

1. **Problem Type**
 - Classification → Softmax output layer, cross-entropy loss
 - Regression → Linear output layer, MSE loss
 - Sequence-to-sequence → Encoder-decoder architecture
 - Image tasks → Convolutional layers
 - Sequential data → Recurrent or transformer layers
 2. **Architecture Components**
 - **Input Layer:** Matches data dimensionality
 - **Hidden Layers:** Feature extraction and transformation
 - **Output Layer:** Task-specific predictions
 3. **Hyperparameters**
 - Learning rate α
 - Batch size
 - Number of layers and neurons per layer
 - Regularization parameters (L_1 , L_2 , dropout)
-

1.5 Training Pipeline

1.5.1 Understanding the Training Process

Training a neural network is an iterative refinement process. We begin with a network of random weights—essentially a network that makes completely random predictions. Through repeated exposure to data, the network gradually learns to make better predictions. Each iteration involves:

1. **Forward Pass:** Feed data through the network to get predictions
2. **Loss Computation:** Calculate how far off those predictions are from the correct answers
3. **Backward Pass:** Calculate which parameter changes would reduce the loss
4. **Update:** Make small adjustments in the right direction

This process repeats thousands of times, with the network constantly improving. Think of it like learning to throw a dart: - First throw: You have no idea where it will land - You observe where it landed vs. where you aimed - You adjust your aim based on this feedback - After many throws, you've unconsciously learned the right angle, force, and release timing

The difference is that neural networks can do this with millions of parameters simultaneously, and the “feedback” comes from a loss function instead of your eyes.

1.5.2 Mini-Batches and Gradient Averaging

One critical principle: we don't update weights after seeing a single example. Instead, we process multiple examples together in **mini-batches** (typically 32-512 examples) and compute average gradients. This approach provides several benefits:

- **Noise Reduction:** A single outlier example won't cause wild weight updates; averaging smooths out individual data quirks
- **Computational Efficiency:** Processing multiple examples together leverages parallelization
- **Better Generalization:** The network learns the general pattern, not specific individual examples

The batch size is a key hyperparameter—too small and updates are noisy; too large and training is slow and may get stuck in poor local minima.

1.5.3 Complete Training Flow

Training Pipeline:

Raw Data → Preprocessing → Tokenization/Normalization → Data Loader → Batch Creation → Forward Pass → Compute Loss → Backward Pass → Update Weights

Training Loop: - Repeat batch processing until epoch complete - After each epoch: validate model - If validation not acceptable: continue training - If validation acceptable: save model

1.5.4 Training Procedure (Pseudocode)



ALGORITHM: TrainNeuralNetwork

INPUT:

- training_data: dataset of (input, target) pairs
- network: neural network with random initialized weights
- epochs: number of training iterations
- learning_rate: step size for gradient descent
- batch_size: number of samples per mini-batch

OUTPUT: trained network

PROCEDURE:

FOR epoch = 1 TO epochs DO

 Shuffle(training_data)

 FOR each batch IN MiniBatches(training_data, batch_size) DO

 // Forward propagation

 predictions = ForwardPass(network, batch.inputs)

 // Compute loss

 loss = ComputeLoss(predictions, batch.targets)

 // Backward propagation

 gradients = BackwardPass(network, loss)

 // Update weights

 FOR each layer IN network DO

 layer.weights = layer.weights - learning_rate * gradients[layer]

 layer.biases = layer.biases - learning_rate * gradients[layer + "_bias"]

 END FOR

 END FOR

 // Validation

 validation_loss = Evaluate(network, validation_data)

 IF validation_loss < best_loss THEN

 SaveModel(network)

 best_loss = validation_loss

 END IF

END FOR

RETURN network

1.5.5 C++ Implementation Skeleton

```
#include <vector>
#include <cmath>
#include <random>

class NeuralNetwork {
private:
    std::vector<std::vector<std::vector<double>>> weights; // weights[layer][neuron][input]
    std::vector<std::vector<double>> biases; // biases[layer][neuron]
    std::vector<std::vector<double>> activations; // activations[layer][neuron]

public:
    NeuralNetwork(const std::vector<int>& layer_sizes) {
        // Initialize random weights and biases
        std::random_device rd;
        std::mt19937 gen(rd());
        std::normal_distribution<> dist(0.0, 1.0);

        for (size_t i = 1; i < layer_sizes.size(); ++i) {
            std::vector<std::vector<double>> layer_weights;
            std::vector<double> layer_biases;

            for (int j = 0; j < layer_sizes[i]; ++j) {
                std::vector<double> neuron_weights;
                for (int k = 0; k < layer_sizes[i-1]; ++k) {
                    neuron_weights.push_back(dist(gen) * std::sqrt(2.0 / layer_sizes[i-1]));
                }
            }
        }
    }
};
```

```

        layer_weights.push_back(neuron_weights);
        layer_biases.push_back(0.0);
    }

    weights.push_back(layer_weights);
    biases.push_back(layer_biases);
}
}

std::vector<double> forward(const std::vector<double>& input) {
    activations.clear();
    activations.push_back(input);

    for (size_t layer = 0; layer < weights.size(); ++layer) {
        std::vector<double> layer_output;

        for (size_t neuron = 0; neuron < weights[layer].size(); ++neuron) {
            double sum = biases[layer][neuron];
            for (size_t i = 0; i < weights[layer][neuron].size(); ++i) {
                sum += weights[layer][neuron][i] * activations[layer][i];
            }
            // Apply activation function (ReLU for hidden layers)
            layer_output.push_back(layer < weights.size() - 1 ? relu(sum) : sum);
        }

        activations.push_back(layer_output);
    }

    return activations.back();
}

void train(const std::vector<std::pair<std::vector<double>, std::vector<double>>>& training_data,
           int epochs, double learning_rate, int batch_size) {
    for (int epoch = 0; epoch < epochs; ++epoch) {
        double total_loss = 0.0;

        for (size_t i = 0; i < training_data.size(); i += batch_size) {
            // Process mini-batch
            std::vector<std::vector<std::vector<double>>> weight_gradients(weights.size());
            std::vector<std::vector<double>> bias_gradients(biases.size());

            // Initialize gradient accumulators
            for (size_t l = 0; l < weights.size(); ++l) {
                weight_gradients[l].resize(weights[l].size());
                bias_gradients[l].resize(biases[l].size(), 0.0);
                for (size_t n = 0; n < weights[l].size(); ++n) {
                    weight_gradients[l][n].resize(weights[l][n].size(), 0.0);
                }
            }

            // Accumulate gradients over batch
            int batch_count = 0;
            for (size_t j = i; j < std::min(i + batch_size, training_data.size()); ++j) {
                auto prediction = forward(training_data[j].first);
                double loss = compute_mse_loss(prediction, training_data[j].second);
                total_loss += loss;

                // Backward pass (gradient computation)
                backward(training_data[j].second, weight_gradients, bias_gradients);
                batch_count++;
            }

            // Update weights with averaged gradients
            for (size_t l = 0; l < weights.size(); ++l) {
                for (size_t n = 0; n < weights[l].size(); ++n) {
                    for (size_t w = 0; w < weights[l][n].size(); ++w) {
                        weights[l][n][w] -= learning_rate * weight_gradients[l][n][w] / batch_count;
                    }
                    biases[l][n] -= learning_rate * bias_gradients[l][n] / batch_count;
                }
            }
        }
    }
}

```

```

// Print epoch statistics
if (epoch % 10 == 0) {
    std::cout << "Epoch " << epoch << ", Loss: " << total_loss / training_data.size() << std::endl;
}
}
}

private:
double relu(double x) { return std::max(0.0, x); }
double relu_derivative(double x) { return x > 0 ? 1.0 : 0.0; }

double compute_mse_loss(const std::vector<double>& prediction,
                        const std::vector<double>& target) {
    double loss = 0.0;
    for (size_t i = 0; i < prediction.size(); ++i) {
        double diff = prediction[i] - target[i];
        loss += diff * diff;
    }
    return loss / prediction.size();
}

void backward(const std::vector<double>& target,
              std::vector<std::vector<std::vector<double>>>& weight_gradients,
              std::vector<std::vector<double>>>& bias_gradients);
// Implementation details in Backpropagation section
};

```

1.6 Forward Propagation

1.6.1 What Forward Propagation Does

Forward propagation is the process of pushing input data through the network layer by layer to produce predictions. It's called "forward" because information flows in one direction: from input through hidden layers to output.

Conceptually, imagine a flow of information: - You feed a photo into the network - The first layer detects simple patterns (edges, colors) - Intermediate layers combine those patterns into more complex features (textures, shapes) - Later layers recognize objects (wheels, windows) - Finally, the output layer decides: "This is a car"

Each layer takes the output from the previous layer and transforms it through two operations: 1. **Linear transformation**: Multiply by weights and add bias 2. **Non-linear activation**: Apply an activation function

1.6.2 Why Two Operations?

The **linear transformation** (weighted sum) is where learning happens—by adjusting weights, the network learns which combinations of inputs matter. But by itself, stacking linear operations would create a network no more powerful than a single-layer network.

The **activation function** adds non-linearity, enabling the network to learn complex curved decision boundaries and relationships. Without it, all layers would collapse into a single linear transformation, defeating the purpose of having multiple layers.

1.6.3 Mathematical Formulation

For a layer with index L , given: - Input activations from previous layer: $\mathbf{a}^{(L-1)}$ - Weight matrix: $\mathbf{W}^{(L)}$ - Bias vector: $\mathbf{b}^{(L)}$

Pre-activation (linear transformation):

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$$

Post-activation (apply non-linearity):

$$\mathbf{a}^{(L)} = \sigma(\mathbf{z}^{(L)})$$

Key definitions: - $\mathbf{z}^{(L)}$: Pre-activation values (weighted sum before activation function) - σ : Activation function (ReLU, sigmoid, tanh, etc.) - $\mathbf{a}^{(L)}$: Post-activation output (layer's contribution to next layer)

1.6.4 Pseudocode

```

FUNCTION ForwardPass(network, input)
  activations[0] = input

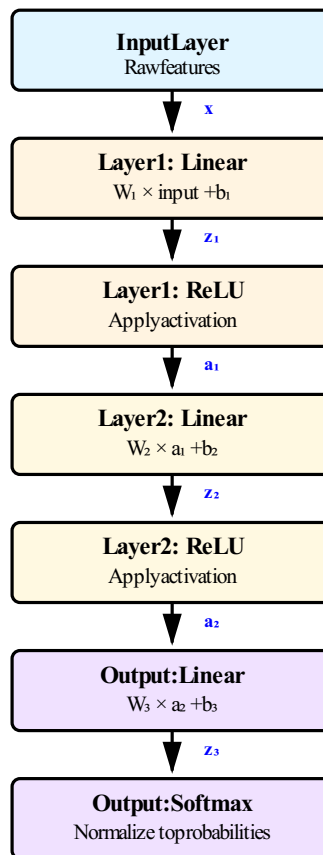
  FOR layer = 1 TO network.num_layers DO
    z = MatrixMultiply(network.weights[layer], activations[layer-1]) + network.biases[layer]
    activations[layer] = ApplyActivation(z, network.activation_functions[layer])
  END FOR

  RETURN activations[network.num_layers]
END FUNCTION

```

1.6.5 Sequence Diagram

ForwardPass Sequence

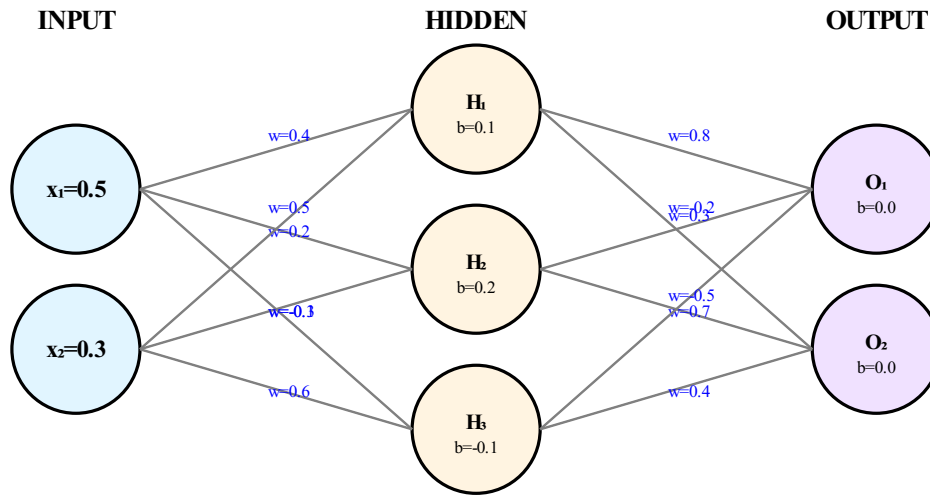


1.6.6 Complete Step-by-Step Example with Numbers

Let's walk through a concrete example of a tiny neural network processing one input. This will show exactly how numbers flow through each operation.

Network Architecture: 2 inputs → 3 hidden neurons → 2 outputs

1.6.6.1 Network Diagram with All Parameters



Architecture Summary: - **Input Layer:** 2 neurons ($x_1=0.5$, $x_2=0.3$) - **Hidden Layer:** 3 neurons with biases $[0.1, 0.2, -0.1]$ - **Output Layer:** 2 neurons with biases $[0.0, 0.0]$ - **Weights:** Fully connected between layers

1.6.6.2 Step 1: Input Layer $\rightarrow$ Hidden Layer (Forward Pass)

Given: - Input: $\mathbf{x} = [0.5, 0.3]$ - Weights $\mathbf{W}^{(1)}$ (3×2 matrix): $W_1 = [[0.4, 0.5], [0.2, -0.1], [-0.3, 0.6]]$ - Biases $\mathbf{b}^{(1)} = [0.1, 0.2, -0.1]$

Computation:

For **Hidden Neuron 1:**

$$z_1^{(1)} = (0.4 \times 0.5) + (0.5 \times 0.3) + 0.1 = 0.2 + 0.15 + 0.1 = 0.45$$

$$a_1^{(1)} = \text{ReLU}(0.45) = \max(0, 0.45) = 0.5$$

For **Hidden Neuron 2:**

$$z_2^{(1)} = (0.2 \times 0.5) + (-0.1 \times 0.3) + 0.2 = 0.1 - 0.03 + 0.2 = 0.27$$

$$a_2^{(1)} = \text{ReLU}(0.27) = 0.3$$

For **Hidden Neuron 3:**

$$z_3^{(1)} = (-0.3 \times 0.5) + (0.6 \times 0.3) + (-0.1) = -0.15 + 0.18 - 0.1 = -0.07$$

$$a_3^{(1)} = \text{ReLU}(-0.07) = 0.0$$

Result: Hidden layer activations = $[0.5, 0.3, 0.0]$

1.6.6.3 Visualization: Hidden Layer Computation

Neuron	Inputs	Computation	Sum	Activation	Status
Neuron 1	$x_1=0.5, x_2=0.3$	$(0.5 \times 0.4) + (0.3 \times 0.5) + 0.1$	$z_1 = 0.45$	$\text{ReLU}(0.45) = \mathbf{0.5}$	☐ ACTIVE
Neuron 2	$x_1=0.5, x_2=0.3$	$(0.5 \times 0.2) + (0.3 \times -0.1) + 0.2$	$z_2 = 0.27$	$\text{ReLU}(0.27) = \mathbf{0.3}$	☐ ACTIVE
Neuron 3	$x_1=0.5, x_2=0.3$	$(0.5 \times -0.3) + (0.3 \times 0.6) + (-0.1)$	$z_3 = -0.07$	$\text{ReLU}(-0.07) = \mathbf{0.0}$	☐ DEAD

Detailed Breakdown:

Neuron 1: $0.20 + 0.15 + 0.1 = 0.45 \rightarrow \text{ReLU} \rightarrow 0.5$

Neuron 2: $0.10 - 0.03 + 0.2 = 0.27 \rightarrow \text{ReLU} \rightarrow 0.3$

Neuron 3: $-0.15 + 0.18 - 0.1 = -0.07 \rightarrow \text{ReLU} \rightarrow 0.0$ (killed by ReLU)

1.6.6.4 Step 2: Hidden Layer → Output Layer

Given: - Hidden activations: $\mathbf{a}^{(1)} = [0.5, 0.3, 0.0]$ - Weights $\mathbf{W}^{(2)}$ (2×3 matrix): $\mathbf{W}_2 = \begin{bmatrix} 0.8 & 0.3 & -0.5 \\ -0.2 & 0.7 & 0.4 \end{bmatrix}$ - Biases $\mathbf{b}^{(2)} = [0.0, 0.0]$

Computation:

For **Output Neuron 1:**

$$z_1^{(2)} = (0.8 \times 0.5) + (0.3 \times 0.3) + (-0.5 \times 0.0) + 0.0$$

$$z_1^{(2)} = 0.4 + 0.09 + 0.0 = 0.49$$

$$a_1^{(2)} = 0.5 \text{ (using sigmoid: } \frac{1}{1 + e^{-0.49}} \approx 0.62 \text{)}$$

For **Output Neuron 2:**

$$z_2^{(2)} = (-0.2 \times 0.5) + (0.7 \times 0.3) + (0.4 \times 0.0) + 0.0$$

$$z_2^{(2)} = -0.1 + 0.21 + 0.0 = 0.11$$

$$a_2^{(2)} = 0.1 \text{ (using sigmoid: } \frac{1}{1 + e^{-0.11}} \approx 0.53 \text{)}$$

Final Network Output: $[0.6, 0.5]$

1.6.6.5 Visualization: Output Layer Computation

Neuron	Inputs	Computation	Sum	Activation	Output
Output 1	$h_1=0.5, h_2=0.3, h_3=0.0$	$(0.5 \times 0.8) + (0.3 \times 0.3) + (0.0 \times -0.5) + 0.0$	$z_1 = 0.49$	Sigmoid(0.49)	≈ 0.6
Output 2	$h_1=0.5, h_2=0.3, h_3=0.0$	$(0.5 \times -0.2) + (0.3 \times 0.7) + (0.0 \times 0.4) + 0.0$	$z_2 = 0.11$	Sigmoid(0.11)	≈ 0.5

Detailed Breakdown:

Output 1: $0.40 + 0.09 + 0.00 + 0.0 = 0.49 \rightarrow \text{Sigmoid} \rightarrow 0.6$

Output 2: $-0.10 + 0.21 + 0.00 + 0.0 = 0.11 \rightarrow \text{Sigmoid} \rightarrow 0.5$

1.6.6.6 Step 3: Loss Calculation

Given: - Network prediction: $\hat{\mathbf{y}} = [0.6, 0.5]$ - True target: $\mathbf{y} = [1.0, 0.0]$ - Using Mean Squared Error (MSE)

Computation:

$$L = \frac{1}{2} \sum_{i=1}^2 (y_i - \hat{y}_i)^2$$

$$L = \frac{1}{2} [(1.0 - 0.6)^2 + (0.0 - 0.5)^2]$$

$$L = \frac{1}{2} [(0.4)^2 + (-0.5)^2]$$

$$L = \frac{1}{2} [0.16 + 0.25] = \frac{0.41}{2} = 0.2$$

Loss value: $L = 0.2$

1.6.6.7 Step 4: Backpropagation (Computing Gradients)

Now we compute how much each weight contributed to the error, flowing backwards through the network.

Output Layer Gradients:

Error signal at output:

$$\delta_1^{(2)} = \frac{\partial L}{\partial z_1^{(2)}} = (0.6 - 1.0) \times \sigma'(0.49) = -0.4 \times 0.24 = -0.1$$

$$\delta_2^{(2)} = \frac{\partial L}{\partial z_2^{(2)}} = (0.5 - 0.0) \times \sigma'(0.11) = 0.5 \times 0.25 = 0.1$$

Weight gradients for output layer:

$$\frac{\partial L}{\partial W_{11}^{(2)}} = \delta_1^{(2)} \times a_1^{(1)} = -0.1 \times 0.5 = -0.05$$

$$\frac{\partial L}{\partial W_{12}^{(2)}} = \delta_1^{(2)} \times a_2^{(1)} = -0.1 \times 0.3 = -0.03$$

$$\frac{\partial L}{\partial W_{21}^{(2)}} = \delta_2^{(2)} \times a_1^{(1)} = 0.1 \times 0.5 = 0.05$$

Hidden Layer Gradients:

Error flowing back to hidden neurons:

$$\begin{aligned}\delta_1^{(1)} &= [(W_{11}^{(2)} \times \delta_1^{(2)}) + (W_{21}^{(2)} \times \delta_2^{(2)})] \times \text{ReLU}'(0.45) \\ \delta_1^{(1)} &= [(0.8 \times -0.1) + (-0.2 \times 0.1)] \times 1 = [-0.08 - 0.02] = -0.1 \\ \delta_2^{(1)} &= [(0.3 \times -0.1) + (0.7 \times 0.1)] \times 1 = [-0.03 + 0.07] = 0.04 \\ \delta_3^{(1)} &= [(-0.5 \times -0.1) + (0.4 \times 0.1)] \times 0 = 0.0 \text{ (dead neuron)}\end{aligned}$$

1.6.6.8 Visualization: Backpropagation Flow

Error Propagation (Backward):

Layer	Neuron	Delta (δ)	Error	Action
Output	O ₁	-0.1	-0.4 (too low)	Increase weights
Output	O ₂	+0.1	+0.5 (too high)	Decrease weights
Hidden	H ₁	-0.1	From outputs	Adjust input weights
Hidden	H ₂	+0.04	From outputs	Adjust input weights
Hidden	H ₃	0.0	Dead neuron	No gradient

Weight Gradients ($\partial L / \partial W$):

Weight Gradient	Action	New Weight ($\eta=0.1$)
$W_{11}^{(2)}$	-0.05 ☐ Increase	0.8 $\rightarrow$ 0.805
$W_{12}^{(2)}$	-0.03 ☐ Increase	Changes
$W_{21}^{(2)}$	+0.05 ☐ Decrease	-0.2 $\rightarrow$ -0.205

Weight Gradient Action New Weight ($\eta=0.1$)

$W_{22}^{(2)}$ +0.04 ☐ Decrease Changes

Flow: Output errors → Hidden deltas → Weight gradients → Weight updates

1.6.6.9 Step 5: Weight Update



Using gradient descent with learning rate $\eta = 0.1$:

$$W_{new} = W_{old} - \eta \times \frac{\partial L}{\partial W}$$

Example updates:

$$W_{11}^{(2)} : 0.8 - (0.1 \times -0.05) = 0.8 + 0.005 = 0.805$$

$$W_{21}^{(2)} : -0.2 - (0.1 \times 0.05) = -0.2 - 0.005 = -0.205$$

Summary of Complete Training Iteration:

Step	Operation	Input	Output
1	Forward: Input→Hidden	[0.5, 0.3]	[0.5, 0.3, 0.0]
2	Forward: Hidden→Output	[0.5, 0.3, 0.0]	[0.6, 0.5]
3	Loss Calculation	pred=[0.6,0.5], target=[1.0,0.0]	Loss=0.2
4	Backward: Output gradients	$\delta^2 = [-0.1, 0.1]$	$\partial L / \partial W^2$
5	Backward: Hidden gradients	$\delta^1 = [-0.1, 0.04, 0.0]$	$\partial L / \partial W^1$
6	Update weights	All $\partial L / \partial W$	New weights

This cycle repeats thousands of times with different training examples, gradually adjusting weights until the network makes accurate predictions.

1.7 Loss Functions

1.7.1 What is a Loss Function?



The loss function is the **objective function that guides learning**. It answers the fundamental question: “How wrong is our prediction?” By quantifying the error, the loss function provides a signal that training can **minimize**.

Different tasks require different loss functions because they encode domain-specific knowledge about what “correct” means:

- **Regression** (predicting continuous values): We care about the magnitude of error
- **Classification** (predicting discrete categories): We care about confidence in the correct class
- **Ranking** (ordering items by relevance): We care about relative ordering

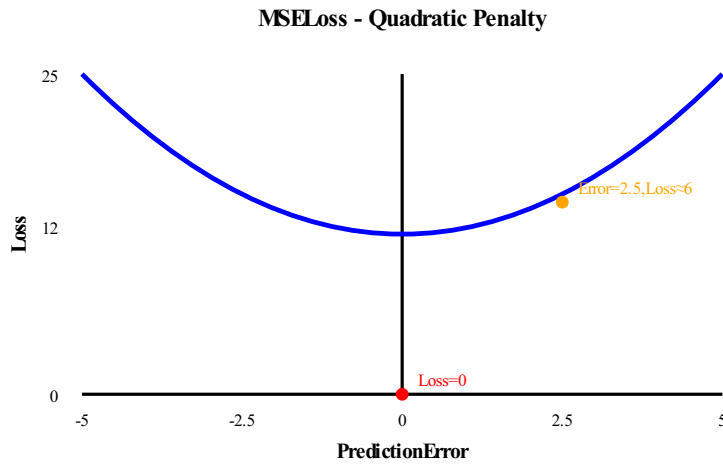
The choice of loss function profoundly affects what the network learns. It’s not just a reporting metric—it actively shapes the learning process.

1.7.2 Mean Squared Error (MSE)

Use Case: Regression tasks where you’re predicting continuous values (prices, temperatures, distances)

Intuition: MSE penalizes large errors much more than small errors (because we square the errors). This makes the network particularly sensitive to outliers and large mistakes.

Visualization - Quadratic Penalty:



Mathematical Formula:

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The summation computes the squared error for each prediction and averages them. Squaring ensures all errors are positive and emphasizes large errors.

Why squared? If we just summed absolute errors, the gradient would be constant (always ± 1), making learning less efficient. The squared term creates a gradient proportional to the error size—small errors produce small gradients (fine-tuning), large errors produce large gradients (aggressive correction).

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Pseudocode:

```
FUNCTION ComputeMSE(predictions, targets)
  sum = 0
  FOR i = 1 TO length(predictions) DO
    error = predictions[i] - targets[i]
    sum = sum + error * error
  END FOR
  RETURN sum / length(predictions)
END FUNCTION
```

C++ Implementation:

```
double compute_mse(const std::vector<double>& predictions,
                  const std::vector<double>& targets) {
    double sum = 0.0;
    for (size_t i = 0; i < predictions.size(); ++i) {
        double error = predictions[i] - targets[i];
        sum += error * error;
    }
    return sum / predictions.size();
}

// Derivative for backpropagation
std::vector<double> mse_derivative(const std::vector<double>& predictions,
                                  const std::vector<double>& targets) {
    std::vector<double> gradients;
    for (size_t i = 0; i < predictions.size(); ++i) {
        gradients.push_back(2.0 * (predictions[i] - targets[i]) / predictions.size());
    }
    return gradients;
}
```

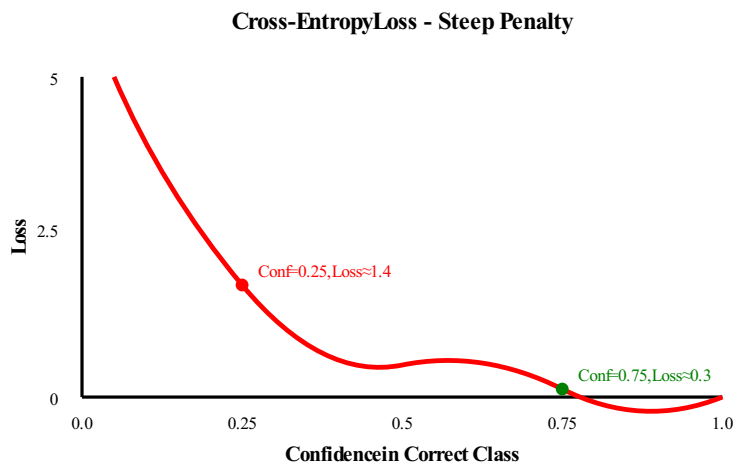
1.7.3 Cross-Entropy Loss

Use Case: Classification tasks where you need to predict which category an item belongs to (spam vs. not spam, digit 0-9, animal species)

Intuition: Cross-entropy measures how “surprised” you’d be by the network’s predictions if the true distribution were revealed. If the network confidently predicts the wrong class, the loss is very high. If it assigns high probability to the correct class, the loss is low.

Why not MSE for classification? While MSE technically works, it treats classification as a regression problem, which doesn’t align with the discrete nature of classes. MSE would treat “predicting probability 0.3 for class A” the same as “predicting 0.3 when the true value is 0,” but in classification we care about whether class A is the *correct* class, not the numerical probability value.

Visualization - Exponential Penalty for Wrong Confident Predictions:



Mathematical Formula (Binary classification):

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Mathematical Formula (Multi-class classification):

$$L_{CCE} = -\sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Where C is the number of classes, $y_{i,c}$ is 1 if sample i has true class c , and $\hat{y}_{i,c}$ is the network’s predicted probability.

Why logarithms? The log function heavily penalizes confident wrong predictions while being forgiving of uncertain predictions. If the network predicts 0.01 probability for the correct class, the loss involves $-\log(0.01)$, which is very high. This forces the network to become confident about correct classes.

Pseudocode:

```
FUNCTION ComputeCrossEntropy(predictions, targets, num_classes)
    loss = 0
    FOR i = 1 TO num_samples DO
        FOR c = 1 TO num_classes DO
            IF targets[i] == c THEN
                loss = loss - log(predictions[i][c])
            END IF
        END FOR
    END FOR
    RETURN loss / num_samples
END FUNCTION
```

```
FUNCTION ComputeCrossEntropy(predictions, targets, num_classes)
    loss = 0
    FOR i = 1 TO num_samples DO
        FOR c = 1 TO num_classes DO
            IF targets[i] == c THEN
                loss = loss - log(predictions[i][c])
            END IF
        END FOR
    END FOR
    RETURN loss / num_samples
END FUNCTION
```

C++ Implementation:

```
double compute_cross_entropy(const std::vector<std::vector<double>>& predictions,
                             const std::vector<int>& targets) {
    double loss = 0.0;
    const double epsilon = 1e-15; // For numerical stability

    for (size_t i = 0; i < predictions.size(); ++i) {
        int target_class = targets[i];
        // Clip predictions to avoid log(0)
        double pred = std::max(epsilon, std::min(1.0 - epsilon, predictions[i][target_class]));
        loss += std::log(pred);
    }

    return loss / predictions.size();
}

// Derivative for softmax + cross-entropy (simplified)
std::vector<std::vector<double>> cross_entropy_softmax_derivative(
    const std::vector<std::vector<double>>& predictions,
    const std::vector<int>& targets) {

    std::vector<std::vector<double>> gradients = predictions; // Copy predictions

    for (size_t i = 0; i < predictions.size(); ++i) {
        int target_class = targets[i];
        for (size_t c = 0; c < predictions[i].size(); ++c) {
            gradients[i][c] = predictions[i][c] - (c == target_class ? 1.0 : 0.0);
        }
    }

    return gradients;
}
```

1.7.4 Huber Loss

Use Case: Robust regression (less sensitive to outliers)

Mathematical Formula:

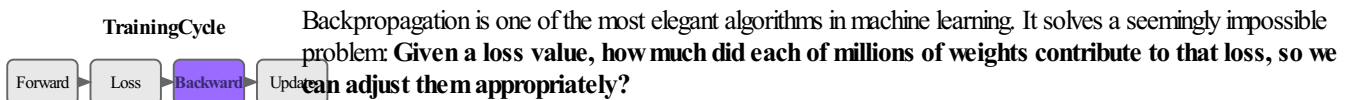
$$\mathcal{L}_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

C++ Implementation:

```
double compute_huber_loss(const std::vector<double>& predictions,
                        const std::vector<double>& targets,
                        double delta = 1.0) {
    double loss = 0.0;
    for (size_t i = 0; i < predictions.size(); ++i) {
        double error = std::abs(predictions[i] - targets[i]);
        if (error <= delta) {
            loss += 0.5 * error * error;
        } else {
            loss += delta * error - 0.5 * delta * delta;
        }
    }
    return loss / predictions.size();
}
```

1.8 Backpropagation Algorithm

1.8.1 Understanding Backpropagation

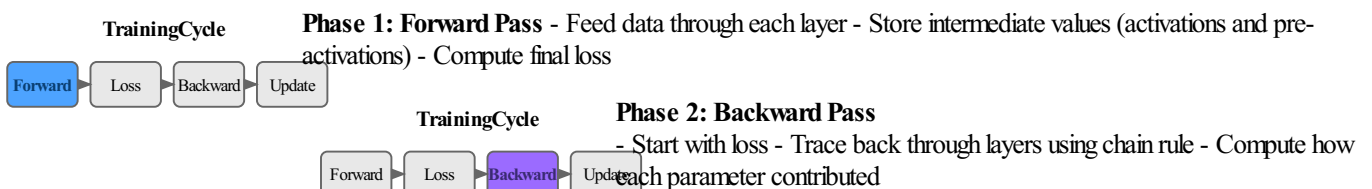


The key insight is **chain rule from calculus**. If the loss depends on the output, and the output depends on layer L , and layer L depends on layer $L - 1$, we can trace how changes in layer $L - 1$ affect the final loss by multiplying these dependency relationships together.

Intuitive example: Imagine a chain of people passing a message: - Person A whispers to Person B who whispers to Person C, who broadcasts the final message - If the final message is wrong, how much is each person to blame? - You measure: What was *Person C's influence* on the final message? - Then: What was *Person B's influence* on what Person C received? - Chain them together: B's blame = C's influence $\times$ B's influence - Continue backward to A

Backpropagation does exactly this for neural networks.

1.8.2 Two-Phase Process



1.8.3 The Delta (Error Signal)

Delta (δ) represents “how much did this layer’s pre-activation contribute to the error?”

For the output layer:

$$\delta^{(L)} = \nabla_{\mathbf{z}^{(L)}} \mathcal{L}(\mathbf{z}^{(L)})$$

$$\delta^{(L)} = \nabla_a L \odot \sigma'(\mathbf{z}^{(L)})$$

For hidden layers:

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

This uses the chain rule: each layer's delta depends on the next layer's delta, weighted by connections, then scaled by activation derivative.

1.8.4 From Deltas to Weight Gradients

Once we have deltas, gradients are:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

Meaning: A weight should change proportional to: - How much the input neuron was activated - How much the output neuron's error was

1.8.5 Mathematical Foundation

For output layer L :

$$\delta^{(L)} = \nabla_a L \odot \sigma'(\mathbf{z}^{(L)})$$

For hidden layer l :

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(\mathbf{z}^{(l)})$$

Gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

Where $\odot$ denotes element-wise multiplication.

1.8.6 Backpropagation Flow

Error Propagation (backward through network): - Output Layer: Compute δ_L - Hidden Layer 2: Propagate δ_{L-1} - Hidden Layer 1: Propagate δ_{L-2} - Input Layer

Weight Updates (at each layer): - Output Layer: Update Weights L using $\partial L / \partial \mathbf{W}_L$ - Hidden Layer 2: Update Weights $L-1$ using $\partial L / \partial \mathbf{W}_{L-1}$ - Hidden Layer 1: Update Weights $L-2$ using $\partial L / \partial \mathbf{W}_{L-2}$

1.8.7 Pseudocode

ALGORITHM: Backpropagation

INPUT:

- network: neural network with current weights
- input: training input
- target: expected output

OUTPUT: gradients for all weights and biases

PROCEDURE:

// Forward pass (save all activations and pre-activations)

activations[0] = input

FOR layer = 1 TO L DO

$z[\text{layer}] = W[\text{layer}] * \text{activations}[\text{layer}-1] + b[\text{layer}]$

$\text{activations}[\text{layer}] = \text{ActivationFunction}(z[\text{layer}])$

END FOR

// Compute output layer error

$\text{delta}[L] = \text{LossDerivative}(\text{activations}[L], \text{target}) * \text{ActivationDerivative}(z[L])$

// Backpropagate error

FOR layer = L-1 DOWN TO 1 DO

$\text{delta}[\text{layer}] = (W[\text{layer}+1]^T * \text{delta}[\text{layer}+1]) * \text{ActivationDerivative}(z[\text{layer}])$

END FOR

// Compute weight and bias gradients

FOR layer = 1 TO L DO

$\text{weight_gradients}[\text{layer}] = \text{delta}[\text{layer}] * \text{activations}[\text{layer}-1]^T$

$\text{bias_gradients}[\text{layer}] = \text{delta}[\text{layer}]$

END FOR

RETURN (weight_gradients, bias_gradients)

1.8.8 C++ Implementation (Complete)

```

void NeuralNetwork::backward(const std::vector<double>& target,
                             std::vector<std::vector<std::vector<double>>>& weight_gradients,
                             std::vector<std::vector<double>>& bias_gradients) {
    // Compute deltas (error signals) for each layer
    std::vector<std::vector<double>> deltas(weights.size());

    // Output layer delta (assuming MSE loss and linear output activation)
    int output_layer = weights.size() - 1;
    deltas[output_layer].resize(activations.back().size());
    for (size_t i = 0; i < activations.back().size(); ++i) {
        // Derivative of MSE: 2(prediction - target)
        deltas[output_layer][i] = 2.0 * (activations.back()[i] - target[i]);
    }

    // Backpropagate through hidden layers
    for (int layer = output_layer - 1; layer >= 0; --layer) {
        deltas[layer].resize(activations[layer + 1].size());

        for (size_t neuron = 0; neuron < activations[layer + 1].size(); ++neuron) {
            double error = 0.0;

            // Sum error from next layer
            for (size_t next_neuron = 0; next_neuron < deltas[layer + 1].size(); ++next_neuron) {
                error += weights[layer + 1][next_neuron][neuron] * deltas[layer + 1][next_neuron];
            }

            // Multiply by activation derivative (ReLU derivative)
            deltas[layer][neuron] = error * relu_derivative(activations[layer + 1][neuron]);
        }
    }

    // Compute weight and bias gradients
    for (size_t layer = 0; layer < weights.size(); ++layer) {
        for (size_t neuron = 0; neuron < weights[layer].size(); ++neuron) {
            // Bias gradient
            bias_gradients[layer][neuron] += deltas[layer][neuron];

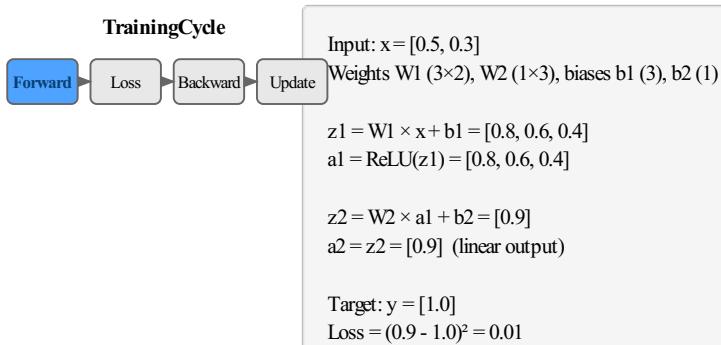
            // Weight gradients
            for (size_t input = 0; input < weights[layer][neuron].size(); ++input) {
                weight_gradients[layer][neuron][input] +=
                    deltas[layer][neuron] * activations[layer][input];
            }
        }
    }
}

```

1.8.9 Detailed Example: 2-Layer Network

Consider a network: Input (2) → Hidden (3) → Output (1)

Forward Pass:



Backward Pass:



$$\delta_2 = d\text{Loss}/da_2 = 2(0.9 - 1.0) = -0.2$$

$$\delta_1 = W_2^T \times \delta_2 \square \text{ReLU}'(z_1)$$

$$= [[w_{21}, w_{22}, w_{23}] \times [-0.2] \square [1, 1, 1]$$

$$= [-0.2 \times w_{21}, -0.2 \times w_{22}, -0.2 \times w_{23}]$$

$$\partial L / \partial W_2 = \delta_2 \times a_1^T = [-0.2] \times [0.8, 0.6, 0.4]$$

$$= [-0.16, -0.12, -0.08]$$

$$\partial L / \partial W_1 = \delta_1 \times x^T \text{ (3} \times 2 \text{ matrix)}$$

1.9 Activation Functions

1.9.1 Why Activation Functions?

Activation functions are the **secret sauce** that gives neural networks their power. Without them, stacking multiple layers would achieve nothing—mathematically, you’d end up with a single-layer network.

The problem without activations: If each layer just does a linear operation (weighted sum), then composing layers together still produces a linear operation:

$$f'(g(x)) = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

This simplifies to a single linear transformation, erasing any benefit from multiple layers.

The solution: Activation functions introduce **non-linearity**, breaking this collapse. By applying ReLU, sigmoid, or tanh after each layer, we enable the network to learn curved, complex decision boundaries and represent intricate functions.

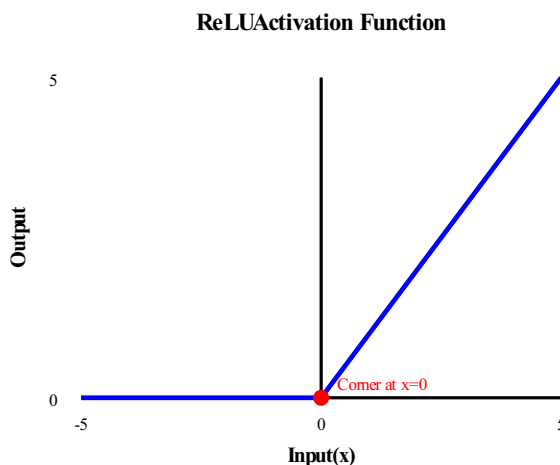
Practical intuition: Think of activation functions as “gates” or “switches” that: - Can turn signals on and off (ReLU) - Can squash values to a specific range (sigmoid, tanh) - Can make the network more expressive by choosing which patterns to activate

1.9.2 ReLU (Rectified Linear Unit)

Formula:

$$\text{ReLU}(x) = \max(0, x)$$

Visualization:



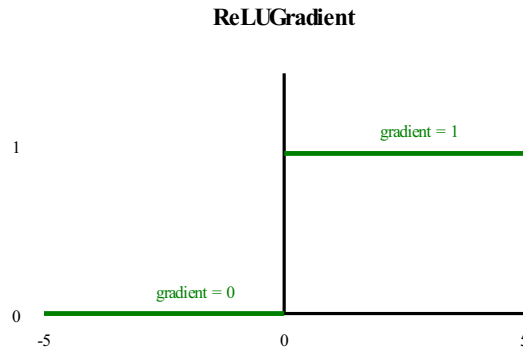
Intuition: ReLU is brutally simple—either pass the signal through (if positive) or kill it (if negative). Despite this simplicity, it’s remarkably effective for learning.

Why it works: - Positive values pass through unchanged (preserving magnitude of useful signals) - Negative values become zero (suppressing irrelevant features) - The function is piecewise linear but non-linear globally (composed across layers) - Computationally efficient (just a comparison and max)

Derivative:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Gradient Visualization:



The derivative is either 1 (full gradient passes) or 0 (no gradient). This simplicity avoids the vanishing gradient problem that plagued earlier sigmoid networks.

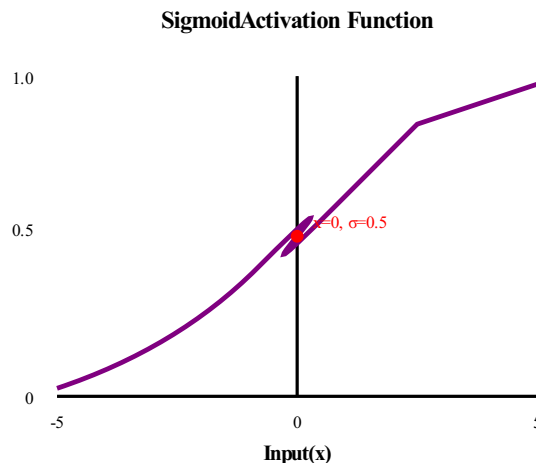
Properties: - **Advantages:** Computationally efficient, mitigates vanishing gradients, induces sparsity - **Disadvantages:** “Dying ReLU” problem (dead neurons that only output 0 forever) - **Use Cases:** Default choice for hidden layers in modern architectures - **Output Range:** $[0, \infty)$

1.9.3 Sigmoid

Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Visualization:



Intuition: Sigmoid is a smooth “S-shaped” curve that squashes any input to a value between 0 and 1. This bounded output is useful for modeling probabilities.

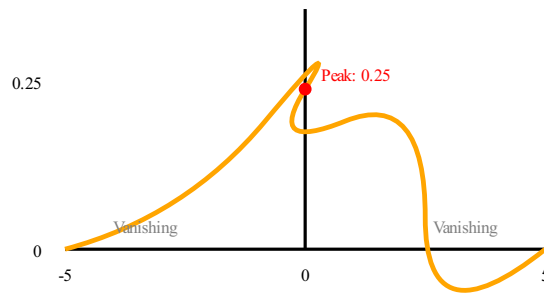
Why sigmoid? The sigmoid function produces outputs interpretable as probabilities—a value of 0.7 naturally means “70% confidence.” This makes it ideal for binary classification where you want a probability of belonging to the positive class.

Derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Gradient Visualization (showing the vanishing gradient problem):

SigmoidGradient (Bell Curve)



Sigmoid gradient forms a bell curve peaking at 0.25 when $x = 0$. For large positive or negative values ($|x| > 3$), gradient drops below 0.02, demonstrating the vanishing gradient problem.

Notice the elegant form: the derivative is just the output times (1 minus output). For values near 0 or 1, this derivative is small (flat regions). Only near $x=0$ is the derivative large (steep gradient).

Properties: - **Output Range:** (0, 1) - **Use Cases:** Binary classification output layer, gate mechanisms in LSTMs - **Problem:** Vanishing gradient—large inputs have near-zero gradients, making deep networks hard to train - **History:** Was the standard activation function before ReLU

C++ Implementation:

```
double sigmoid(double x) {
    // Numerically stable version
    if (x >= 0) {
        return 1.0 / (1.0 + std::exp(-x));
    } else {
        double exp_x = std::exp(x);
        return exp_x / (1.0 + exp_x);
    }
}

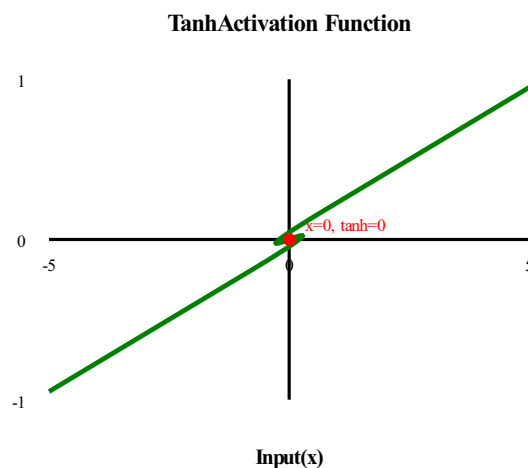
double sigmoid_derivative(double sigmoid_output) {
    return sigmoid_output * (1.0 - sigmoid_output);
}
```

1.9.4 Tanh (Hyperbolic Tangent)

Formula:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Visualization (Zero-centered unlike sigmoid):



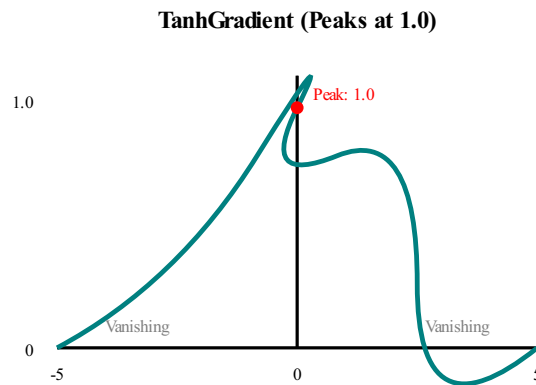
Intuition: Tanh is similar to sigmoid but ranges from -1 to +1 instead of 0 to 1. The key advantage: it's **zero-centered**, meaning the average output is near 0.

Why zero-centered matters: In training, when you average gradients over a batch, if all activations are positive (as with sigmoid), gradients all point in the same direction in weight space, causing inefficient “zig-zagging” during optimization. Tanh’s zero-centered outputs produce more efficient gradient flow.

Derivative:

$$\tanh'(x) = 1 - \tanh^2(x)$$

Gradient Visualization (Stronger gradients than sigmoid):



Properties: - **Output Range:** (-1, 1) - **Advantages:** Zero-centered output (more efficient gradient flow than sigmoid) - **Use Cases:** RNN hidden states (LSTMs, GRUs), old-style hidden layers - **Disadvantage:** Still suffers from vanishing gradients for large $|x|$

C++ Implementation:

```
double tanh_activation(double x) {  
    return std::tanh(x);  
}  
  
double tanh_derivative(double tanh_output) {  
    return 1.0 - tanh_output * tanh_output;  
}
```

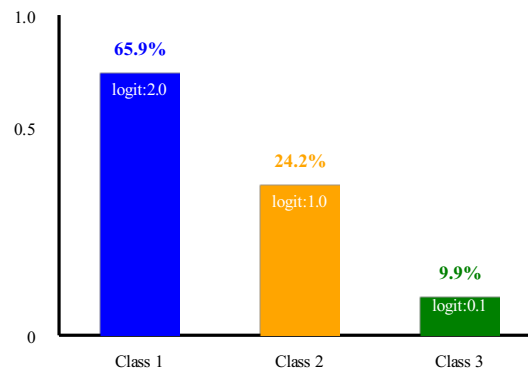
1.9.5 Softmax

Formula (for vector $\mathbf{z}$):

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Visualization (Converting logits [2.0, 1.0, 0.1] to probabilities):

Softmax: Logits → Probabilities



Intuition: Softmax converts arbitrary output values into a **probability distribution**. Each output becomes a probability between 0 and 1, and all probabilities sum to 1.

Why softmax? For multi-class classification (not just binary), we need to express confidence across multiple possible categories. Softmax achieves this elegantly: - Large logits → high probability - Small logits → low probability
- Always produces valid probabilities - Emphasizes the largest logit (winner-take-most behavior)

Example: If raw network outputs are [2.0, 1.0, 0.1] for three classes: - Softmax gives approximately [0.66, 0.24, 0.10] - The largest output (2.0) gets the highest probability - But non-maximum outputs still contribute (unlike argmax)

Properties: - **Output:** Probability distribution (sums to 1) - **Use Cases:** Multi-class classification output layer - **Combined With:** Cross-entropy loss (natural mathematical pair) - **Gradient:** Elegant interaction with cross-entropy—when combined, gradient is simply (prediction - target)

Pseudocode:

```
FUNCTION Softmax(logits)
  // Numerical stability: subtract max
  max_logit = max(logits)
  exp_logits = []
  sum = 0

  FOR i = 1 TO length(logits) DO
    exp_val = exp(logits[i] - max_logit)
    exp_logits[i] = exp_val
    sum = sum + exp_val
  END FOR

  FOR i = 1 TO length(exp_logits) DO
    exp_logits[i] = exp_logits[i] / sum
  END FOR

  RETURN exp_logits
END FUNCTION
```

C++ Implementation:

```

std::vector<double> softmax(const std::vector<double>& logits) {
    // Find max for numerical stability
    double max_logit = *std::max_element(logits.begin(), logits.end());

    std::vector<double> exp_values;
    double sum = 0.0;

    for (double logit : logits) {
        double exp_val = std::exp(logit - max_logit);
        exp_values.push_back(exp_val);
        sum += exp_val;
    }

    for (double& val : exp_values) {
        val /= sum;
    }

    return exp_values;
}

// Jacobian matrix for softmax (simplified for backprop)
std::vector<std::vector<double>> softmax_jacobian(const std::vector<double>& softmax_output) {
    size_t n = softmax_output.size();
    std::vector<std::vector<double>> jacobian(n, std::vector<double>(n));

    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < n; ++j) {
            if (i == j) {
                jacobian[i][j] = softmax_output[i] * (1.0 - softmax_output[i]);
            } else {
                jacobian[i][j] = -softmax_output[i] * softmax_output[j];
            }
        }
    }

    return jacobian;
}

```

Properties: - **Output:** Probability distribution (sums to 1) - **Use Cases:** Multi-class classification output layer - **Note:** Often combined with cross-entropy loss for simplified gradient computation

1.9.6 Leaky ReLU

The Problem ReLU Solves (and Leaky ReLU Solves Better)

Standard ReLU has a subtle but critical flaw: the “**dying ReLU**” problem. Once a neuron receives negative inputs, it outputs 0 and **its gradient is 0**. During backpropagation, this means no weight updates—the neuron is permanently “dead,” contributing nothing to learning.

How does a neuron die? 1. Random initialization gives the neuron some weights 2. Forward pass: input × weights produces negative value → ReLU outputs 0 3. Backprop: gradient is 0 → weight update is 0 (no learning) 4. Next pass: same weights → same negative value → same death 5. Result: Dead neuron forever

In large networks, 30-50% of neurons can die, wasting computation and representational power.

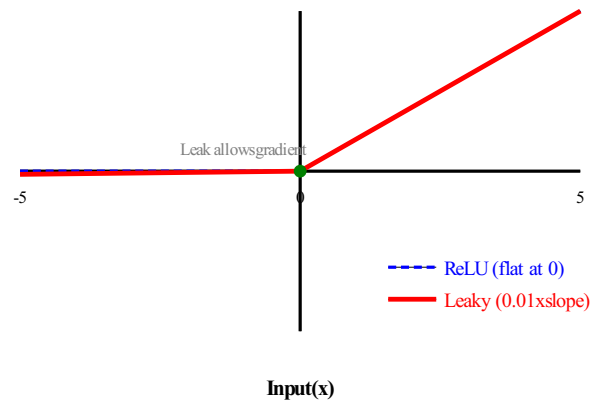
Leaky ReLU solution: Instead of completely killing negative inputs, allow a small leak through.

Formula:

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$$

Visualization (with $\alpha = 0.01$ vs standard ReLU):

LeakyReLU vs Standard ReLU



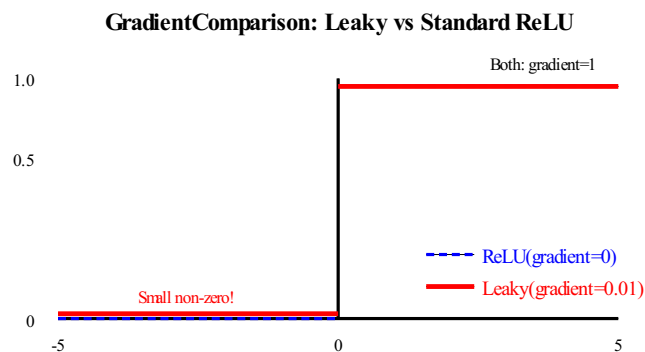
Where α is a small constant (typically 0.01). This means negative values aren't annihilated—they're scaled down by a factor of 100.

Why this fixes the problem: - Negative inputs now produce non-zero output - Non-zero output means non-zero gradient during backpropagation - Weights can still update, allowing the neuron to "recover" from negative inputs - Sparsity benefit (some outputs remain 0) is partially preserved

Derivative (the key for backprop):

$$\text{LeakyReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{otherwise} \end{cases}$$

Gradient Comparison (Leaky ReLU prevents dead neurons):



Note: Even when output is suppressed ($\alpha = 0.01$), the gradient is still $\alpha = 0.01$, not 0. This non-zero gradient allows learning to continue.

C++ Implementation:

```
double leaky_relu(double x, double alpha = 0.01) {
    return x > 0.0 ? x : alpha * x;
}

double leaky_relu_derivative(double x, double alpha = 0.01) {
    return x > 0.0 ? 1.0 : alpha;
}
```

Properties: - **Advantages:** Prevents dying ReLU problem, maintains gradient flow for negative inputs - **Disadvantages:** Slightly more computation than ReLU; hyperparameter α to tune - **Use Cases:** Recommended when dying ReLU observed; large networks especially vulnerable - **Output Range:** $(-\infty, \infty)$ (unbounded like ReLU) - **Common Values:** $\alpha = 0.01$ (default), sometimes 0.1 or 0.3 for more aggressive leaking

1.10 Tokenization

□ **Multimodal Tokenization:** This section covers text tokenization for language models. For comprehensive coverage of **image tokenization** (ViT patches, VQ-VAE, latent diffusion) and **audio tokenization** (mel spectrograms, Whisper, neural audio codecs, music generation), see [A06 — Tokenization Deep Dive](#).

1.10.1 Why Tokenization Matters

Tokenization is the bridge between raw text and neural networks. Neural networks can't directly process words or characters at arbitrary scale—they operate on fixed-size numerical vectors. Tokenization solves this by breaking text into **tokens** (basic units), each mapped to an integer ID that the network can embed and process.

The challenge: We can't use character-level tokens because: - Vocabulary too small: loses meaning of word structure - Vocabulary too large: wasteful if using one token per unique word - Solution: Use subword tokens that capture meaningful patterns

Ideal tokenization: - Handles unknown words gracefully - Captures linguistic structure - Keeps vocabulary manageable (10K-100K tokens) - Efficient for both common and rare concepts

1.10.2 Byte Pair Encoding (BPE)

Core Idea: Build a vocabulary by **iteratively merging the most frequent adjacent pairs** of bytes/tokens. Start with characters, find "ll" occurs most often, merge it to "ll", then find "th" occurs most often, merge it, and so on.

Why it works: - Frequent combinations (like "th" in English) become single tokens - Rare combinations stay as characters - Naturally balances between word-level and character-level granularity - Language-agnostic (works for any writing system)

Example process:

```
Initial: "hello world" → ['h','e','l','l','o',' ','w','o','r','l','d']
Iteration 1: 'll' most frequent → ['h','e','ll','o',' ','w','o','r','l','d']
Iteration 2: 'o ' most frequent → ['h','e','ll','o_',' ','w','o','r','l','d']
Iteration 3: 'ol' most frequent → ['h','e','ll','o_w','o','r','l','d']
Result: Mixed characters and subword units
```

1.10.3 WordPiece (used by BERT)

Variation of BPE with a key difference: When merging, WordPiece prioritizes pairs that **appear frequently AND together** (using likelihood instead of just frequency).

Algorithm: 1. Start with character vocabulary + special tokens 2. Score each pair by: frequency × probability of being a merged unit 3. Merge highest-scoring pair 4. Repeat until desired vocabulary size

Advantage over BPE: More linguistically meaningful merges because it considers contextual relationships, not just raw frequency.

1.10.4 SentencePiece

Novel approach: Train directly on raw bytes (or Unicode codepoints), enabling completely language-agnostic tokenization without preprocessing.

Key features: - No pre-processing needed (no spaces, punctuation handling) - Treats space as a special token (easier reversal) - Works directly on Unicode - Language-universal

Why this matters: Previous tokenizers assumed English-like space-separated words. SentencePiece handles CJK languages (Chinese, Japanese, Korean) naturally without requiring external segmentation.

```

ALGORITHM: TrainBPE
INPUT: corpus (text), num_merges
OUTPUT: vocabulary of subword tokens

PROCEDURE:
// Initialize with characters
vocabulary = GetAllCharacters(corpus)

FOR merge = 1 TO num_merges DO
    // Count all adjacent pairs
    pair_counts = CountPairs(corpus, vocabulary)

    // Find most frequent pair
    best_pair = argmax(pair_counts)

    // Merge the pair and add to vocabulary
    new_token = Concatenate(best_pair)
    vocabulary.add(new_token)

    // Replace all occurrences in corpus
    corpus = ReplacePair(corpus, best_pair, new_token)
END FOR

RETURN vocabulary
END ALGORITHM

FUNCTION TokenizeWithBPE(text, vocabulary)
    tokens = []

    WHILE text is not empty DO
        // Find longest matching token from vocabulary
        longest_match = FindLongestMatch(text, vocabulary)
        tokens.append(longest_match)
        text = RemovePrefix(text, longest_match)
    END WHILE

    RETURN tokens
END FUNCTION

```

C++ Implementation (Simplified):

```

#include <string>
#include <vector>
#include <unordered_map>
#include <algorithm>

class BPETokenizer {
private:
    std::vector<std::string> vocabulary;
    std::unordered_map<std::string, int> token_to_id;

public:
    BPETokenizer(const std::vector<std::string>& vocab) : vocabulary(vocab) {
        for (size_t i = 0; i < vocab.size(); ++i) {
            token_to_id[vocab[i]] = i;
        }
    }

    std::vector<int> encode(const std::string& text) {
        std::vector<int> token_ids;
        std::string remaining = text;

        while (!remaining.empty()) {
            // Greedy longest match
            int best_length = 0;
            std::string best_token;

            for (const auto& token : vocabulary) {
                if (remaining.substr(0, token.length()) == token &&

```

```

        token.length() > best_length) {
            best_length = token.length();
            best_token = token;
        }
    }

    if (best_length == 0) {
        // Unknown token - use special token or skip
        remaining = remaining.substr(1);
        continue;
    }

    token_ids.push_back(token_to_id[best_token]);
    remaining = remaining.substr(best_length);
}

return token_ids;
}

std::string decode(const std::vector<int>& token_ids) {
    std::string text;
    for (int id : token_ids) {
        if (id >= 0 && id < vocabulary.size()) {
            text += vocabulary[id];
        }
    }
    return text;
}

// Pair counting for BPE training
std::unordered_map<std::string, int> count_pairs(const std::vector<std::string>& tokens) {
    std::unordered_map<std::string, int> pairs;

    for (size_t i = 0; i < tokens.size() - 1; ++i) {
        std::string pair = tokens[i] + " " + tokens[i + 1];
        pairs[pair]++;
    }

    return pairs;
}

std::string find_best_pair(const std::unordered_map<std::string, int>& pairs) {
    std::string best_pair;
    int max_count = 0;

    for (const auto& [pair, count] : pairs) {
        if (count > max_count) {
            max_count = count;
            best_pair = pair;
        }
    }

    return best_pair;
}
};

```

1.10.5 WordPiece Tokenization

Difference from BPE: Uses likelihood-based scoring rather than frequency.

Scoring Function:

$$\text{score}(token_1, token_2) = \frac{\text{count}(token_1, token_2)}{\text{count}(token_1) \times \text{count}(token_2)}$$

C++ Implementation Sketch:

```

class WordPieceTokenizer {
private:
    std::unordered_map<std::string, int> vocabulary;
    std::string unknown_token = "[UNK]";
    int max_input_chars = 100;

public:
    std::vector<std::string> tokenize(const std::string& text) {
        std::vector<std::string> output_tokens;

        // Split on whitespace
        std::istringstream iss(text);
        std::string word;

        while (iss >> word) {
            if (word.length() > max_input_chars) {
                output_tokens.push_back(unknown_token);
                continue;
            }

            // Try to tokenize word with WordPiece
            std::vector<std::string> sub_tokens = tokenize_word(word);
            output_tokens.insert(output_tokens.end(), sub_tokens.begin(), sub_tokens.end());
        }

        return output_tokens;
    }

private:
    std::vector<std::string> tokenize_word(const std::string& word) {
        std::vector<std::string> tokens;
        size_t start = 0;

        while (start < word.length()) {
            size_t end = word.length();
            std::string cur_substr;
            bool found = false;

            // Find longest matching subword
            while (start < end) {
                std::string substr = word.substr(start, end - start);
                if (start > 0) {
                    substr = "##" + substr; // WordPiece continuation prefix
                }

                if (vocabulary.find(substr) != vocabulary.end()) {
                    cur_substr = substr;
                    found = true;
                    break;
                }
                end--;
            }

            if (!found) {
                return {unknown_token};
            }

            tokens.push_back(cur_substr);
            start = end;
        }

        return tokens;
    }
};

```

1.10.6 SentencePiece

Key Feature: Language-agnostic, treats text as raw byte stream.

Advantages: - No pre-tokenization required - Works with any language - Reversible tokenization

C++ Usage Example (using SentencePiece library):

```
#include <sentencepiece_processor.h>

class SentencePieceTokenizer {
private:
    sentencepiece::SentencePieceProcessor processor;

public:
    bool load_model(const std::string& model_file) {
        const auto status = processor.Load(model_file);
        return status.ok();
    }

    std::vector<int> encode(const std::string& text) {
        std::vector<int> ids;
        processor.Encode(text, &ids);
        return ids;
    }

    std::string decode(const std::vector<int>& ids) {
        std::string text;
        processor.Decode(ids, &text);
        return text;
    }

    std::vector<std::string> encode_as_pieces(const std::string& text) {
        std::vector<std::string> pieces;
        processor.Encode(text, &pieces);
        return pieces;
    }
};
```

1.10.7 Embedding Layer

After tokenization, tokens are converted to dense vectors:

Pseudocode:

```
FUNCTION TokenEmbedding(token_id, embedding_matrix)
    // embedding_matrix: vocabulary_size × embedding_dim
    RETURN embedding_matrix[token_id]
END FUNCTION
```

C++ Implementation:

```

class EmbeddingLayer {
private:
    std::vector<std::vector<double>> embeddings; // vocab_size × embedding_dim

public:
    EmbeddingLayer(int vocab_size, int embedding_dim) {
        // Initialize with random values
        std::random_device rd;
        std::mt19937 gen(rd());
        std::normal_distribution<> dist(0.0, 1.0);

        embeddings.resize(vocab_size);
        for (int i = 0; i < vocab_size; ++i) {
            embeddings[i].resize(embedding_dim);
            for (int j = 0; j < embedding_dim; ++j) {
                embeddings[i][j] = dist(gen) * 0.01;
            }
        }
    }

    std::vector<double> get_embedding(int token_id) {
        if (token_id >= 0 && token_id < embeddings.size()) {
            return embeddings[token_id];
        }
        return std::vector<double>(embeddings[0].size(), 0.0); // Return zero vector
    }

    std::vector<std::vector<double>> embed_sequence(const std::vector<int>& token_ids) {
        std::vector<std::vector<double>> embedded;
        for (int id : token_ids) {
            embedded.push_back(get_embedding(id));
        }
        return embedded;
    }

    void update_embedding(int token_id, const std::vector<double>& gradient, double learning_rate) {
        for (size_t i = 0; i < embeddings[token_id].size(); ++i) {
            embeddings[token_id][i] -= learning_rate * gradient[i];
        }
    }
};

```

1.11 Inference Flow

1.11.1 Why Inference Is Different From Training

Inference is the moment where your trained network meets the real world. Unlike training—where we compute gradients and update weights—inference is purely **forward propagation**: push data through the network and get predictions.

Key differences that matter:

1. **No Gradient Computation**
 - Training: Forward pass → Loss → Backward pass (compute $\partial L / \partial w$)
 - Inference: Forward pass only → Output (no backprop needed)
 - Implication: ~2x faster per sample, much less memory needed
 - Optimization: Can use **inference-only model formats** that discard gradient history
2. **Determinism vs Randomness**
 - Training: Dropout enabled (randomly removes neurons for regularization)
 - Inference: Dropout disabled (want consistent predictions)
 - Implication: Same input → same output during inference
 - Exception: LLMs sometimes use **temperature** to control randomness
3. **Batch vs Single-Sample**
 - Training: Often processes mini-batches for efficiency
 - Inference: Often processes one sample at a time (streaming)
 - Implication: Need to handle variable batch sizes

- Optimization: Can batch multiple inference requests for efficiency gain

4. Why Latency Matters

- Training happens once offline
- Inference happens for every user request
- 100ms delay → poor user experience
- Optimization: Model quantization, pruning, specialized hardware (GPUs, TPUs)

1.11.2 Batch Processing for Efficiency

The paradox: Single inference is wasteful, but users expect fast responses.

Solution - Batch processing: Collect multiple inference requests, process them together:

```
Request 1 → Queue
Request 2 → Queue (batch size = 3)
Request 3 → Queue
↓
Process all 3 together: 1 forward pass, 3 results
↓
Return to users (minimal additional latency)
```

Why matrices matter here: A single image is a $1 \times 3 \times 224 \times 224$ tensor; 32 images is $32 \times 3 \times 224 \times 224$. Matrix multiplication on GPU is **highly efficient** for large batches. The overhead doesn't scale linearly.

1.11.3 Output Transformation

From network output to user result:

1. **Raw logits:** [2.3, -1.1, 0.8] (unbounded numbers)
2. **Softmax:** [0.66, 0.10, 0.24] (probabilities)
3. **Argmax:** Class index 0 (prediction)
4. **User-friendly:** "This is a dog with 66% confidence"

Temperature parameter (for LLMs): - Temperature = 0.1 (sharp): Model picks highest probability (deterministic) - Temperature = 1.0 (normal): Standard softmax probabilities - Temperature = 2.0 (spread): More random, creative outputs - Formula: $\text{softmax}(\text{logits} / \text{temperature})$

1.11.4 Inference Pipeline

Inference Flow:

Raw Input → Preprocessing → Tokenization → Embedding → Forward Pass → Output Processing → Prediction

1.11.5 Pseudocode


```
ALGORITHM: Inference
INPUT: model, input_data
OUTPUT: predictions

PROCEDURE:
// Preprocessing
normalized_input = Normalize(input_data)

// Tokenization (for text)
IF input is text THEN
    token_ids = Tokenize(normalized_input)
    embeddings = GetEmbeddings(token_ids)
    model_input = embeddings
ELSE
    model_input = normalized_input
END IF

// Forward pass (no gradient computation)
predictions = model.forward(model_input)

// Post-processing
IF task is classification THEN
    predicted_class = argmax(predictions)
    RETURN predicted_class
ELSE IF task is regression THEN
    RETURN predictions
END IF
END PROCEDURE
```

1.11.6 C++ Inference Example

```

class InferenceEngine {
private:
    NeuralNetwork model;
    BPETokenizer tokenizer;
    EmbeddingLayer embeddings;

public:
    InferenceEngine(const NeuralNetwork& trained_model,
                    const BPETokenizer& tok,
                    const EmbeddingLayer& emb)
        : model(trained_model), tokenizer(tok), embeddings(emb) {}

    std::vector<double> predict(const std::string& text) {
        // Tokenize input
        std::vector<int> token_ids = tokenizer.encode(text);

        // Get embeddings
        std::vector<std::vector<double>> embedded_tokens = embeddings.embed_sequence(token_ids);

        // Flatten embeddings for input (or use sequence processing)
        std::vector<double> model_input = flatten_embeddings(embedded_tokens);

        // Forward pass (inference mode - no gradient tracking)
        std::vector<double> output = model.forward(model_input);

        return output;
    }

    int predict_class(const std::string& text) {
        std::vector<double> logits = predict(text);

        // Apply softmax
        std::vector<double> probabilities = softmax(logits);

        // Return class with highest probability
        return std::distance(probabilities.begin(),
                             std::max_element(probabilities.begin(), probabilities.end()));
    }

    std::vector<std::pair<int, double>> predict_top_k(const std::string& text, int k) {
        std::vector<double> logits = predict(text);
        std::vector<double> probabilities = softmax(logits);

        // Create pairs of (class_id, probability)
        std::vector<std::pair<int, double>> class_probs;
        for (size_t i = 0; i < probabilities.size(); ++i) {
            class_probs.emplace_back(i, probabilities[i]);
        }

        // Sort by probability descending
        std::partial_sort(class_probs.begin(),
                          class_probs.begin() + k,
                          class_probs.end(),
                          [](const auto& a, const auto& b) { return a.second > b.second; });

        return std::vector<std::pair<int, double>>(class_probs.begin(),
                                                  class_probs.begin() + k);
    }

private:
    std::vector<double> flatten_embeddings(const std::vector<std::vector<double>>& embedded) {
        std::vector<double> flattened;
        for (const auto& emb : embedded) {
            flattened.insert(flattened.end(), emb.begin(), emb.end());
        }
        return flattened;
    }
};

```

1.11.7 Batch Inference

For efficient processing of multiple inputs:

```
class BatchInferenceEngine {
public:
    std::vector<std::vector<double>>> batch_predict(
        const std::vector<std::string>& texts,
        int batch_size = 32) {

        std::vector<std::vector<double>>> all_predictions;

        for (size_t i = 0; i < texts.size(); i += batch_size) {
            size_t end = std::min(i + batch_size, texts.size());

            // Process batch
            std::vector<std::vector<double>>> batch_inputs;
            for (size_t j = i; j < end; ++j) {
                batch_inputs.push_back(preprocess(texts[j]));
            }

            // Forward pass for entire batch (matrix operations)
            std::vector<std::vector<double>>> batch_outputs =
                model.forward_batch(batch_inputs);

            all_predictions.insert(all_predictions.end(),
                                  batch_outputs.begin(),
                                  batch_outputs.end());
        }

        return all_predictions;
    }
};
```

1.12 Matrix Implementation Notes

1.12.1 Why Matrices Instead of Loops?

The core realization: Neural networks are fundamentally matrix operations. Why does this matter?

Reason 1: Computational Efficiency - Naive approach: Loop through each neuron, multiply weights, sum inputs (slow) - Matrix approach: Exploit BLAS (Basic Linear Algebra Subroutines) - BLAS is highly optimized after decades of scientific computing - GPU hardware (CUDA, Metal) has specialized matrix multiplication circuits - Result: 100-1000x speedup for large batches

Reason 2: Hardware Acceleration - Modern GPUs are designed for matrix operations - NVIDIA CUDA, Apple Metal, AMD ROCm all excel at parallel matrix math - Even CPUs have SIMD (Single Instruction Multiple Data) for vectorization - A single GPU operation (matrix multiply) can beat 1000 CPU loop iterations

Reason 3: Memory Efficiency - Matrices fit in cache-friendly layouts (row-major or column-major) - CPU prefetching works well with predictable memory access patterns - Scattered, loop-based access causes cache misses - Modern CPUs can read an entire matrix row in a few cycles with prefetching

Reason 4: Batch Processing Power - Single sample: 1×784 input for MNIST handwriting - Batch of 32 samples: 32×784 input matrix - The matrix multiplication overhead is nearly the same for 1 or 32 samples - This is why batching is so powerful

1.12.2 Weight Matrix Representation

For a layer with n_{in} inputs and n_{out} outputs:

$$\mathbf{W} \in \mathbb{R}^{n_{out} \times n_{in}}$$

Why this shape? The output will be n_{out} values, so we need a weight matrix with n_{out} rows. Each row contains n_{in} weights for combining the inputs.

Forward pass for batch:

$$\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

Where: - $\mathbf{X} \in \mathbb{R}^{n_{in} \times batch_size}$: Input matrix (each column is one sample) - $\mathbf{Z} \in \mathbb{R}^{n_{out} \times batch_size}$: Pre-activation output - $\mathbf{b} \in \mathbb{R}^{n_{out} \times 1}$: Bias vector (broadcasted to all samples)

Memory layout considerations: - Row-major (C/C++ default): Iterate rows first for cache efficiency - Column-major (Fortran, BLAS): Matrices stored column-by-column - BLAS libraries expect specific layouts; mismatch kills performance

1.12.3 Practical Optimization Tradeoffs

Computation vs Memory: - Larger batch size $\rightarrow$ better GPU utilization (faster) - Larger batch size $\rightarrow$ more memory needed - Too large $\rightarrow$ out-of-memory error - Sweet spot: Use largest batch that fits in VRAM

Precision vs Speed: - Full precision (float64): Accurate but slow - Single precision (float32): Default, good balance - Half precision (float16): 2x faster, slight accuracy loss - Integer (int8): 4x faster for inference, training trickier - Modern GPUs have specialized float16/int8 cores

What BLAS libraries do: - Detect your hardware (CPU type, GPU presence) - Choose optimal algorithm (different matrix sizes need different approaches) - Parallelize across cores/GPUs - Optimize memory access patterns - Handle numerical stability

1.12.4 Matrix Operations in C++

```
#include <vector>

// Matrix multiplication: C = A * B
std::vector<std::vector<double>>> matrix_multiply(
    const std::vector<std::vector<double>>>& A,
    const std::vector<std::vector<double>>>& B) {

    size_t m = A.size();    // rows of A
    size_t n = B[0].size(); // cols of B
    size_t k = A[0].size(); // cols of A (must equal rows of B)

    std::vector<std::vector<double>>> C(m, std::vector<double>(n, 0.0));

    for (size_t i = 0; i < m; ++i) {
        for (size_t j = 0; j < n; ++j) {
            for (size_t p = 0; p < k; ++p) {
                C[i][j] += A[i][p] * B[p][j];
            }
        }
    }

    return C;
}

// Matrix transpose: B = A^T
std::vector<std::vector<double>>> transpose(
    const std::vector<std::vector<double>>>& A) {

    size_t m = A.size();
    size_t n = A[0].size();

    std::vector<std::vector<double>>> At(n, std::vector<double>(m));

    for (size_t i = 0; i < m; ++i) {
        for (size_t j = 0; j < n; ++j) {
            At[j][i] = A[i][j];
        }
    }

    return At;
}

// Element-wise multiplication (Hadamard product): C = A □ B
std::vector<std::vector<double>>> hadamard_product(
    const std::vector<std::vector<double>>>& A,
    const std::vector<std::vector<double>>>& B) {

    size_t m = A.size();
```

```

size_t n = A[0].size();

std::vector<std::vector<double>>> C(m, std::vector<double>(n));

for (size_t i = 0; i < m; ++i) {
    for (size_t j = 0; j < n; ++j) {
        C[i][j] = A[i][j] * B[i][j];
    }
}

return C;
}

// Matrix addition: C = A + B
std::vector<std::vector<double>>> matrix_add(
    const std::vector<std::vector<double>>& A,
    const std::vector<std::vector<double>>& B) {

    size_t m = A.size();
    size_t n = A[0].size();

    std::vector<std::vector<double>>> C(m, std::vector<double>(n));

    for (size_t i = 0; i < m; ++i) {
        for (size_t j = 0; j < n; ++j) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    return C;
}

```

1.12.5 Batch Processing Example

```

class BatchLayer {
public:
    std::vector<std::vector<double>>> weights; // n_out × n_in
    std::vector<double> biases; // n_out

    // Forward pass for entire batch
    std::vector<std::vector<double>>> forward(
        const std::vector<std::vector<double>>& inputs) {
        // inputs: n_in × batch_size
        // output: n_out × batch_size

        // Z = W × X
        auto Z = matrix_multiply(weights, inputs);

        // Add bias to each column (batch element)
        for (size_t i = 0; i < Z.size(); ++i) {
            for (size_t j = 0; j < Z[i].size(); ++j) {
                Z[i][j] += biases[i];
            }
        }

        return Z;
    }
};

```

1.12.6 Performance Considerations

Note: These implementations use naive algorithms for clarity. Production systems would use:

1. **Optimized BLAS libraries:**
 - Intel MKL
 - OpenBLAS

- Apple Accelerate
- 2. **Cache-friendly layouts:**
 - Row-major vs. column-major storage
 - Blocking/tiling for cache locality
- 3. **SIMD instructions:**
 - AVX, AVX-512 for vectorization
 - Compiler auto-vectorization
- 4. **Parallel execution:**
 - OpenMP for CPU parallelism
 - Thread pools for batch processing
- 5. **GPU acceleration** (future elaboration):
 - CUDA for NVIDIA GPUs
 - ROCm for AMD GPUs
 - Metal for Apple Silicon
 - Vulkan/OpenCL for cross-platform

Example: Using BLAS for Matrix Multiplication:

```
// Using CBLAS (not implementing here, just showing the interface)
#include <blas.h>

void optimized_matrix_multiply(
    const double* A, const double* B, double* C,
    int m, int n, int k) {

    // C = A * B
    // A: m x k, B: k x n, C: m x n
    cblas_dgemm(
        CblasRowMajor, // Row-major storage
        CblasNoTrans,   // Don't transpose A
        CblasNoTrans,   // Don't transpose B
        m, n, k,        // Dimensions
        1.0,            // Alpha (scaling factor)
        A, k,            // Matrix A and its leading dimension
        B, n,            // Matrix B and its leading dimension
        0.0,            // Beta (for C = alpha*AB + beta*C)
        C, n            // Matrix C and its leading dimension
    );
}
```

1.13 References

1.13.1 Academic Papers

1. **Backpropagation:**
 - Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). "Learning representations by back-propagating errors." *Nature*, 323(6088), 533-536.
2. **Activation Functions:**
 - Glorot, X., Bordes, A., & Bengio, Y. (2011). "Deep Sparse Rectifier Neural Networks." *AISTATS*.
 - Nair, V., & Hinton, G. E. (2010). "Rectified linear units improve restricted boltzmann machines." *ICML*.
3. **Tokenization:**
 - Sennrich, R., Haddow, B., & Birch, A. (2016). "Neural Machine Translation of Rare Words with Subword Units." *ACL*.
 - Kudo, T., & Richardson, J. (2018). "SentencePiece: A simple and language independent approach to subword tokenization." *EMNLP*.

1.13.2 Books

1. **Deep Learning** by Ian Goodfellow, Yoshua Bengio, and Aaron Courville
 - URL: <https://www.deeplearningbook.org/>
2. **Neural Networks and Deep Learning** by Michael Nielsen
 - URL: <http://neuralnetworksanddeeplearning.com/>
3. **Pattern Recognition and Machine Learning** by Christopher Bishop

1.13.3 Online Resources

1. **Stanford CS231n**: Convolutional Neural Networks for Visual Recognition
 - URL: <http://cs231n.stanford.edu/>
2. **3Blue1Brown Neural Network Series**:
 - URL: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
3. **Distill.pub**: Interactive ML Visualizations
 - URL: <https://distill.pub/>

1.13.4 Implementation Libraries

1. **Eigen** (C++ linear algebra library):
 - URL: <https://eigen.tuxfamily.org/>
2. **Intel MKL** (Math Kernel Library):
 - URL: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html>
3. **SentencePiece** (Tokenization library):
 - URL: <https://github.com/google/sentencepiece>

Navigation: [Back to Index](#) | [Next: ONNX Standard](#)